CodeAstra 2.0: Media Forensics Platform — Problem, Solution & Build Plan

Oct 12, 2026 · @ranger

Executive summary

For the AI/ML track of CodeAstra 2.0, the team is building against **Problem 2: Multimodal Synthetic-Media Forensics & Provenance Reasoning**. It beat the other two AI/ML options on demo credibility, depth and alignment with how the judges said they want the problem solved.

The platform turns "Fake, 92%" into evidence: separate visual, audio, synchronization, metadata and provenance findings, fused into one final assessment with its supporting evidence, limitations and confidence. To keep that working live on longer or higher-resolution files, the pipeline screens cheaply across a whole clip first and spends expensive analysis only where the screening flags something unusual.

This document covers the problem statement, why it was chosen, the solution and output format, the system architecture, the adaptive-analysis addition, the build workflow for a 4-person team, the 24-hour timeline, and the buffer, stretch features, risks and fallbacks.

The problem statement

Track: AI/ML · **Title:** Multimodal Synthetic-Media Forensics & Provenance Reasoning

Context

Synthetic audio, video and imagery are increasingly used in misinformation, fraud, impersonation and digital investigations. A simple real-or-fake classification is not enough for forensic work, because investigators need to know what evidence suggests manipulation, where inconsistencies occur, how the media was processed, and whether its claimed origin can be trusted. A file may also be only partially manipulated, such as genuine footage with a synthetic voice, a replaced region, altered timestamps, or several generations of compression and editing.

Core challenge

Build a multimodal forensic investigation system that treats media as a collection of interconnected evidence rather than a binary classification problem. The system should correlate visual inconsistencies and manipulation artifacts, audio characteristics and

synthetic-speech indicators, audio-video synchronization, frame-level and temporal anomalies, metadata and encoding history, compression and re-encoding patterns, file modification history, source and device information where available, cryptographic hashes and provenance records, and the relationships between multiple copies or versions of the same media. It should generate an evidence-backed forensic assessment that shows why particular portions of the media are suspicious and how pieces of evidence support or contradict each other.

Expected 24-hour prototype

A working vertical slice: media ingestion, forensic analysis, evidence correlation, timeline or region analysis, provenance reconstruction, and investigator review. The prototype should include multimodal media ingestion, multiple independent forensic checks, frame or segment-level anomaly identification, audio-video consistency analysis, metadata and encoding inspection, evidence correlation with confidence estimation, a visual forensic timeline or suspicious-region representation, provenance or hash verification, and human investigator review and annotation.

Important requirement

The solution must not depend primarily on an external deepfake-detection API or third-party fake-or-real classifier. External pretrained models or forensic libraries, if used, must function as individual evidence sources within the larger reasoning pipeline, not as the final decision-maker.

Convergence

Cybersecurity and blockchain can anchor hashes and provenance; IoT can provide trusted capture-device identity; web provides the investigator workspace.

Why this problem is win-worthy

Problem	What you build	Strength	Weakness	Rank
2. Media Forensics & Provenance	Multi-check forensic pipeline, evidence fusion, provenance and investigator review	Live demo on unseen files; explicit judging signal; spans signal processing, CV, audio, crypto	Individual detectors can misfire on unseen files	1

Problem	What you build	Strength	Weakness	Rank
1. Urban Incident Fusion	Clustering, spatio-temporal correlation, Bayesian or rule-based causal layer, dashboard	Low-risk engineering; strong out-of-order and duplicate-signal demo	Runs on synthetic data you generated; "causal" reads as heuristics	2
3. Operations Copilot	RAG with citations, contradiction detection, recommendations	Most accessible to build	Most teams will pick it; demos look alike; wins only on polish	3

Four things set Problem 2 apart:

- **Live verification:** a judge can hand you a file and watch the system analyze it, unlike a demo on data you made up.
- **Explicit judging signal:** the requirement not to depend on an external classifier tells you the judges reward reasoning over API calls.
- **Depth:** it combines signal processing, computer vision, audio, container forensics, cryptography and evidence fusion.
- **Convergence:** it connects naturally to blockchain hash anchoring, trusted capture devices and a web workspace.

The main risk is that individual detectors are unreliable, and a live demo on an unseen file can misfire. The brief is framed around evidence correlation and showing why, not detection accuracy, so a system that honestly reports "inconclusive, here is the conflicting evidence" can still score well.

Our solution and output format

A typical detector returns "Fake, 92%". The platform instead returns one finding per piece of evidence, each tied to a place in the media, and combines them into a final assessment with limitations and confidence.

Finding type	Example output	Evidence it cites
Visual	A face region has unusual blending artifacts in frames 120 to 145.	Frame range, region box, heatmap, noise-residual and quality-mismatch scores

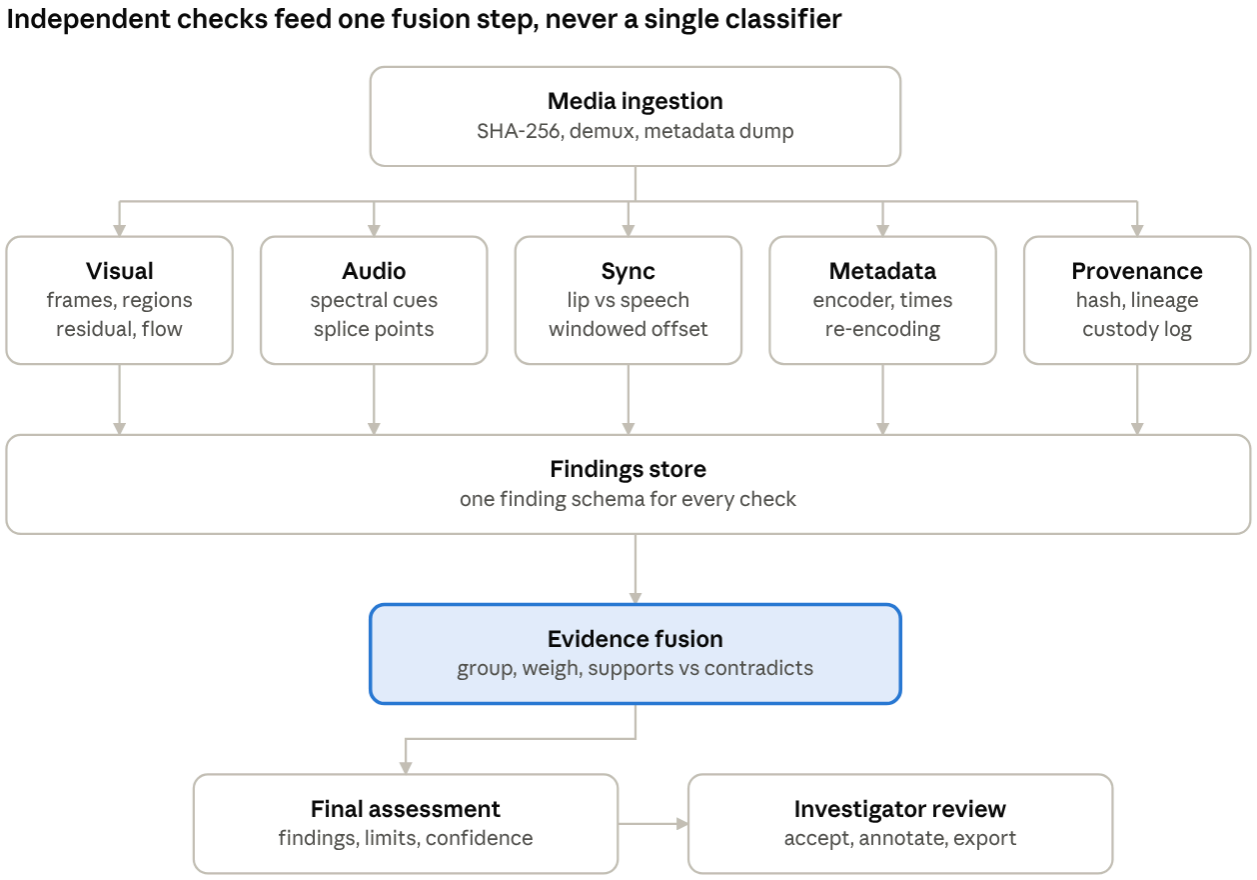
Finding type	Example output	Evidence it cites
Audio	A speech segment has synthetic-speech indicators between 00:08 and 00:12.	Time range, spectrogram crop, high-frequency cutoff and spectral-flatness values
Synchronization	Audio and lip movement appear inconsistent during a particular interval.	Interval, estimated offset, correlation drop, offset curve
Metadata	The file contains encoding information consistent with a later export or re-encoding.	Field names and values, timestamp conflicts, encoder strings
Provenance	The SHA-256 hash matches, or does not match, a previously recorded version.	Both hashes, the changed segments from the Merkle tree, the lineage graph
Final assessment	Findings combined with supporting evidence, limitations and confidence.	Links to every finding, the contradictions, an overall state

Every check emits the same schema, so fusion and the UI never need to know which check produced a finding:

```
{
  "id": "vis-003",
  "modality": "visual | audio | sync | metadata | provenance",
  "check": "face_blend_residual",
  "segment": { "start": 120, "end": 145, "unit": "frame" },
  "region": [x, y, w, h],
  "score": 0.0,
  "reliability": 0.0,
  "stance": "supports_manipulation | contradicts | neutral | not_assessable",
  "rationale": "plain-language reason",
  "limitations": "what could make this wrong"
}
```

The final assessment has one of three states: likely manipulated, likely authentic, or inconclusive. Inconclusive is a legitimate answer when evidence conflicts or checks are not assessable.

System architecture



pipeline · 5 parallel checks, 1 fusion step

Each check runs in parallel and writes findings in the same schema. Fusion is the only step that weighs them, so no single detector decides the verdict.

What to add to win

1. Build your own ground-truth test set first

Make 6 to 8 files you fully control, so the system's explanations and a results table can be shown directly: a clean clip; a clip with a TTS-cloned voice swapped in (Piper or Coqui); a spliced or replaced region; a clip re-encoded 3 times; a clip with edited metadata and timestamps; two copies derived from each other.

2. Evidence fusion that preserves uncertainty

- Use Bayesian log-odds or Dempster-Shafer fusion, with per-check reliability weights.
- Include an explicit "insufficient evidence" state, so the system can abstain.
- Draw supports and contradicts edges in an evidence graph.

- Add a "remove this evidence and see how the verdict changes" counterfactual toggle. It is a strong demo moment and shows the reasoning is real.

3. Independent checks across layers

Layer	Checks
Container	ffprobe and exiftool encoder tags; creation vs modification time; audio/video duration mismatch; edit lists; odd GOP structure
Compression	Double-compression and I-frame spacing anomalies; face-region vs background quality mismatch
Visual	ELA; noise-residual inconsistency; optical-flow jumps; frame duplication or drops
Audio	Vocoder-style high-frequency cutoffs; noise-floor and room-tone discontinuities at splice points; optionally a pretrained anti-spoofing model such as AASIST as one evidence source
Audio-video sync	Mouth aperture from MediaPipe Face Mesh, cross-correlated with the audio envelope over sliding windows to estimate offset per segment

4. Provenance that is cryptographically meaningful

- Hash the file per segment into a Merkle tree, so you can prove which segment was altered.
- Use perceptual hashing to build a lineage graph of copies and versions.
- Read C2PA manifests if present (c2patool).
- Keep a hash-chained, Ed25519-signed chain-of-custody log, including investigator annotations.
- Anchor the root hash on a testnet or a simple tamper-evident ledger.
- For IoT convergence, simulate a capture device (even a phone or ESP32 camera) that signs the hash at capture, so the verifier can show "origin chain intact" or "broken".

5. Investigator workspace

Show swimlane timelines (visual, audio, sync, metadata, compression). Clicking a suspicious segment jumps to the frame with a heatmap box. Investigators can accept or reject evidence, add annotations, and export a PDF report with the chain of custody.

6. An "attack mode" button

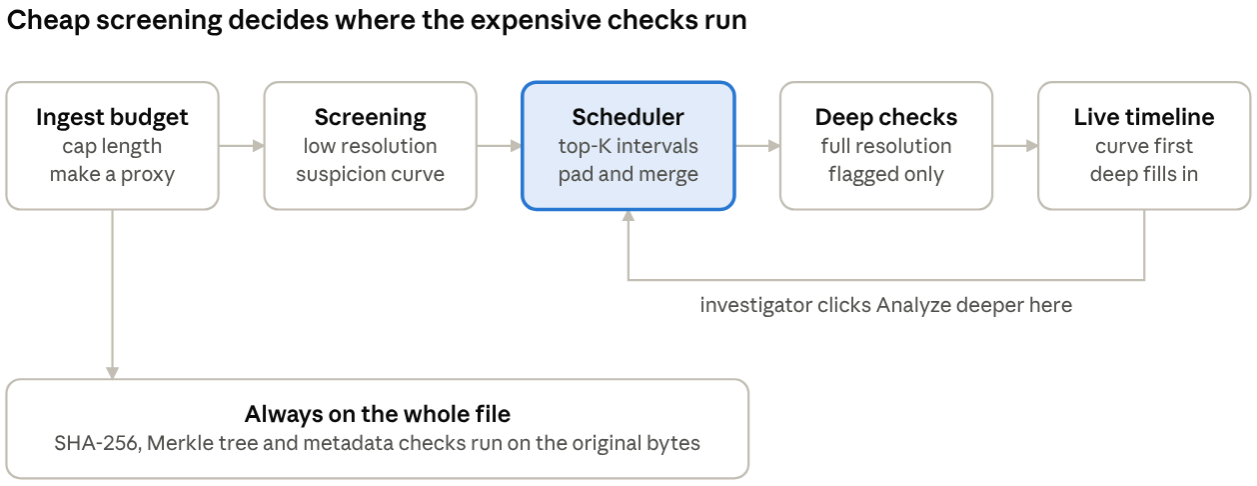
Re-compress, resize or add noise to a file and show the evidence degrading and the confidence dropping honestly. Judges rarely see teams demonstrate their system's limits, and it signals rigor.

7. Grounded explanations

If an LLM is used, let it write the summary only from structured evidence and never decide the verdict. Fusion makes the call, which keeps the system inside the "no external classifier as decision-maker" rule.

Adaptive forensic analysis

Video processing, face detection, audio analysis and fusion can exceed demo time, especially on longer or higher-resolution files. The answer is to limit input length and resolution, and use lightweight screening to decide where detailed analysis is needed: scan at low resolution, find an unusual interval, then analyze that interval at full resolution. This makes the platform practical for live demonstrations and longer videos, and is a differentiator since most teams will run every check on every frame and then demo on a short clip.



adaptive analysis flow · 5 stages, 1 feedback loop

Screening runs over the whole clip and the scheduler spends the deep-check budget on the flagged intervals. The investigator can send the scheduler back to any interval, and the cheap integrity checks never depend on the proxy.

Budget at ingest. Cap demo clips at a fixed length and make a downscaled proxy for analysis. Keep the original untouched: SHA-256, Merkle tree and metadata checks must run on the original bytes, and they are cheap enough to always run on the whole file.

Screening pass. Compute low-cost signals over the whole clip into a per-modality suspicion curve: frame-difference spikes, noise-residual variance and a cheap face-present flag for video; spectral flux, high-frequency energy drops and noise-floor jumps for audio.

Scheduler. Pick the top-K suspicious intervals, pad them, merge overlaps, and run the expensive checks only there: full-resolution ELA, optical flow, Face Mesh sync, and AASIST if used. The time budget sets K, so a longer clip gets the same wall-clock time with less coverage instead of a timeout.

Progressive UI. The screening curve appears in seconds on the timeline, then deep findings fill in as workers finish. An "Analyze deeper here" button lets the investigator trigger deep analysis on any interval, which fits the human-review requirement and makes a strong live-demo moment.

Keeping it honest. Screening can miss subtle manipulation. Add `depth: screen | deep` to the finding schema and a coverage figure to the final assessment, so the report states what percentage was analyzed in depth. Deep-check a few random, unflagged control intervals to produce a rough miss-rate estimate for the results table, and report the real time saving as a number measured on the test clips rather than a claimed figure.

This costs roughly 3 to 4 hours of the Backend lead's time in the H6 to H12 window alongside fusion, since the screeners can reuse features the deep checks already compute. Treat it as core, not a buffer item, since it protects the demo against slow files.

Build workflow

Four roles for a 4-person team. Each module ends with a defined output in the finding schema, so modules can be built and tested independently.

- **Video lead:** all visual and frame-level checks.
- **Audio lead:** audio manipulation checks and audio-video sync.
- **Backend lead:** ingestion, metadata, provenance, fusion, adaptive scheduling, API.
- **Frontend lead:** investigator workspace, timeline, report export.

Ingestion and test set (Backend lead with everyone, H0 to H4)

1. Build the upload endpoint, store the original read-only, and compute its SHA-256 on ingest.
2. Demux with ffmpeg: sampled frames, a 16 kHz mono audio track, a full metadata dump, and a downscaled proxy for the adaptive screening pass.
3. Build a job runner that fires every check in parallel and writes findings to one store.
4. Create the test set: a clean clip, a TTS-cloned voice swap, a spliced region, a clip re-encoded 3 times, edited metadata and timestamps, and two derived copies.

Output: a job record with frames, audio, a proxy and an empty findings table, plus the six test files.

Video manipulation checks (Video lead, H1 to H15)

1. Sample frames and detect face regions (MediaPipe or OpenCV).
2. Compute error-level analysis and noise-residual maps per frame.
3. Compare face-region quality against the background and look for blending seams.
4. Detect duplicated or dropped frames and optical-flow jumps.
5. Detect double compression through I-frame spacing and GOP anomalies.

Output: visual findings with frame range, region box and heatmap, run on the scheduler's flagged intervals.

Audio manipulation checks (Audio lead, H1 to H15)

1. Normalise the audio and compute spectrograms.
2. Score synthetic-speech indicators: high-frequency cutoff, spectral flatness, vocoder-style artifacts. A pretrained anti-spoofing model such as AASIST can join as one evidence source.
3. Detect splices through noise-floor and room-tone discontinuities.
4. Optional: compare speaker embeddings across segments to catch a swapped voice.

Output: audio findings with time range and a spectrogram crop.

Audio-video synchronization (Audio lead, H8 to H16)

1. Measure mouth aperture per frame with MediaPipe Face Mesh.
2. Align the audio energy envelope to the video frame rate.
3. Cross-correlate over sliding windows to estimate the audio-video offset per window.
4. Flag windows where the offset drifts or the correlation drops. Mark windows with no face or no speech as not assessable.

Output: sync findings per interval and an offset curve for the timeline.

Metadata and encoding inspection (Backend lead, H2 to H10)

1. Dump container and codec details with ffprobe and exiftool.
2. Check timestamps: creation vs modification, audio vs video duration, edit lists.
3. Flag re-encoding traces: editing-software encoder strings and signs of several generations.
4. Read a C2PA manifest if one is present (c2patool).

Output: metadata findings that quote the exact field names and values.

Provenance (Backend lead, H8 to H17)

1. Compare the whole-file SHA-256 against a ledger of previously recorded versions.
2. Hash the file per segment into a Merkle tree to show which segment changed.
3. Use perceptual hashing to build a lineage graph of copies and versions.
4. Keep a hash-chained, Ed25519-signed chain-of-custody log, including investigator actions.
5. Simulate a capture device that signs the hash at capture, so the verifier can show the origin chain as intact or broken.

Output: a match or mismatch finding, the changed segments, and a lineage graph.

Adaptive screening and scheduler (Backend lead, H6 to H12)

1. Build the low-resolution screening pass for video and audio into a suspicion curve.
2. Build the scheduler that picks top-K intervals, pads and merges them within the time budget.
3. Wire the "Analyze deeper here" trigger from the frontend into the scheduler.
4. Add the `depth` and coverage fields to the finding schema.

Output: a suspicion curve, a flagged-interval list, and deep findings restricted to those intervals.

Fusion and final assessment (Backend lead, H6 to H17)

1. Normalise every check into the finding schema.
2. Group findings that overlap in time or region.
3. Fuse with Bayesian log-odds or Dempster-Shafer, using per-check reliability weights and an explicit "insufficient evidence" state.

4. Build the evidence graph with supports and contradicts edges, plus a "remove this evidence" counterfactual toggle.
5. Generate the final assessment. If an LLM writes the wording, it works only from structured evidence and never decides the verdict.

Output: the final assessment with findings, limitations and confidence.

Investigator workspace (Frontend lead, H1 to H17)

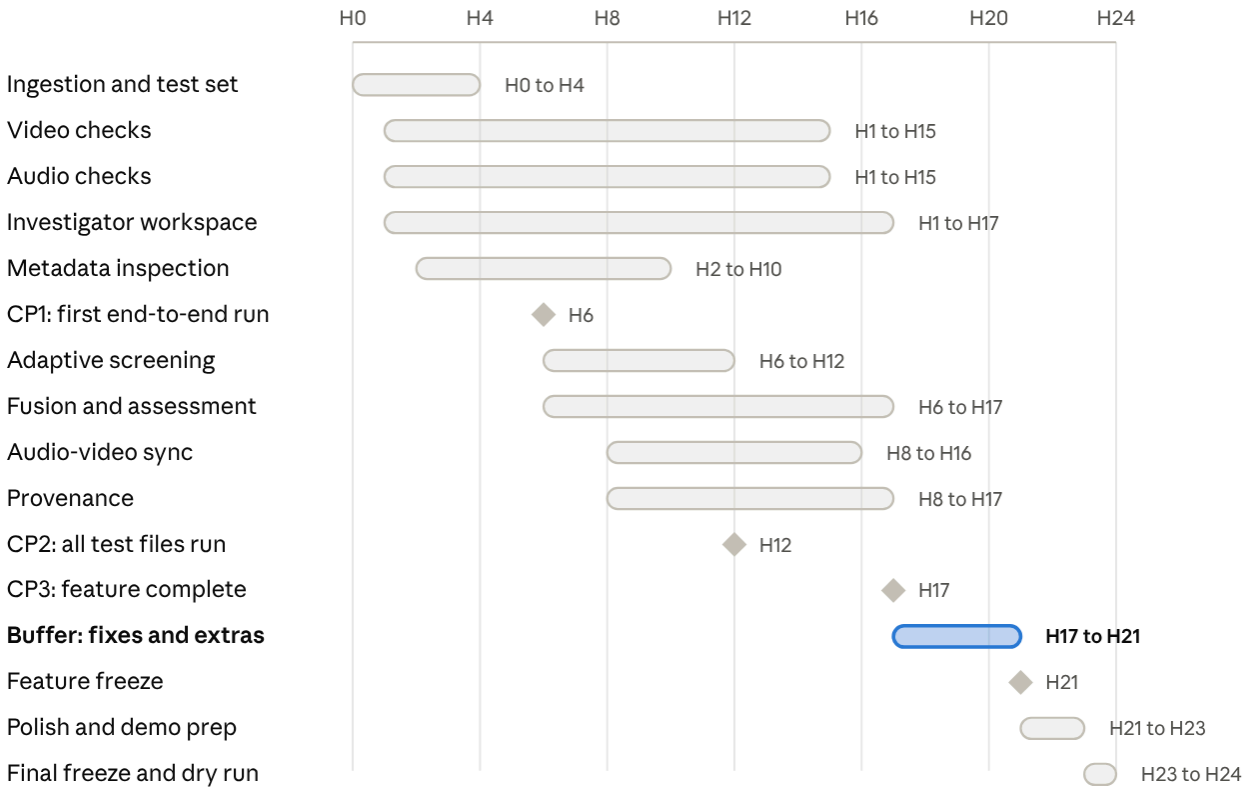
1. Build upload and job status.
2. Build a swimlane timeline (visual, audio, sync, metadata, provenance) beside a video player, with the screening curve showing first and deep findings filling in.
3. Make a click on a finding jump to the frame and draw the heatmap box.
4. Add accept or reject and annotation on each piece of evidence, logged in the custody chain.
5. Export a PDF report that includes the chain of custody.

Output: a clickable investigator review flow that matches the findings schema.

24-hour timeline

The 4-hour buffer sits between feature-complete and the freeze

Hours from the start of the hackathon, H0 to H24



24-hour build timeline · 12 workstreams, 3 checkpoints

Every module is built thin by CP1 and deepened after, so a slipped module shrinks the demo instead of blocking it. Diamonds are checkpoints and the freeze; the highlighted bar is the buffer.

Buffer and stretch features

The buffer is H17 to H21, four hours with no planned work. Spend it in this order: first fix anything that slipped from CP3, then add stretch features from the menu below, and stop adding at the H21 feature freeze. A feature that is not working by H21 is cut, not rushed into the demo.

Priority	Stretch feature	Why it helps	Rough cost
1	Attack mode button (re-compress, resize or add noise, then show the evidence degrading and confidence dropping honestly)	Shows the limits of the system and signals rigor	1 to 1.5 h
2	Counterfactual toggle polish ("remove this evidence and see how the verdict changes")	Proves the reasoning is real, not a lookup	1 h
3	Results table on the test set, with the system's explanation of each file and the screening miss-rate estimate	Backs every claim with numbers on files you control	1 h
4	Capture-device signing simulation (a phone or ESP32 camera signs the hash at capture)	Delivers the IoT convergence story	1.5 h
5	Blockchain anchoring of the root hash on a testnet or tamper-evident ledger	Delivers the cybersecurity and blockchain convergence story	1.5 h
6	Speaker-embedding consistency check	Catches a swapped voice that spectral checks miss	1.5 h
7	Multi-copy lineage view comparing several versions of one media file	Strengthens provenance reasoning	1 h

Leave room for a new function thought of during the build: the only rule is that it emits the shared finding schema, so it plugs into fusion and the timeline without changes elsewhere.

Risks and fallbacks

Each module has a cut-down version that still emits valid findings, so a slipped module degrades the demo instead of breaking it.

Risk	Early warning	Fallback
A visual or audio detector misfires on unseen files	False positives on the clean test clip at CP2	Lower its reliability weight and report "inconclusive" instead of dropping the check
Audio-video sync is unstable	No usable offset on most windows by H12	Run it only on clips with a clear frontal face and mark other windows not assessable
Provenance work overruns	Merkle tree not working by H14	Ship whole-file SHA-256 match plus the signed custody log, and add per-segment hashing in the buffer
Adaptive scheduler misses real manipulation	Control-interval spot checks show a high miss rate at H12	Raise K or shrink the clip cap so more of the file gets deep analysis
Frontend timeline is late	Timeline not showing real findings at CP2	Render a simple findings list with frame and time links, then upgrade to swimlanes
A schema change breaks the integration	Any check failing to load at CP1	Freeze the schema in the first hour; additions only, never renames
Demo fails live on an unseen file	Dry run at H22 fails	Prepare a recorded backup run and fall back to the six prepared test files